Oracle PL/SQL Triggers – Lab Manual

Prepared By: Kinza

December 7, 2025

Contents

1 Introduction

A trigger in Oracle PL/SQL is a stored program that automatically executes in response to specific events on a table or database. Triggers help implement business rules, audit changes, enforce data integrity, and automate tasks.

This manual covers:

- DML Triggers
- DDL Triggers
- System Triggers
- Instead-of Triggers
- Auditing Triggers
- Login / Email Notification Triggers
- Time-zone Setting Trigger

2 Types of Triggers

- **DML Triggers** – Fire on INSERT, UPDATE, DELETE.
- **DDL Triggers** – Fire when schema objects are created, altered, or dropped.
- **System Triggers** – Fire at the database level (startup, shutdown, login).
- **Instead-of Triggers** – Used on Views.
- **Compound Triggers** – Provide multi-timing control.

3 DML Trigger Examples

3.1 BEFORE INSERT Validation Trigger

```
CREATE OR REPLACE TRIGGER validate_salary
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Salary cannot be negative.');
    END IF;
END;
/
```

Demonstration

```
-- Valid salary
INSERT INTO employees VALUES (101, 'Ali', 50000);

-- Invalid salary (TRIGGER WILL FIRE ERROR)
INSERT INTO employees VALUES (102, 'Sara', -9000);
-- Output: ORA-20001: Salary cannot be negative
```

3.2 AFTER UPDATE Audit Trigger

```
CREATE TABLE emp_audit (
    emp_id       NUMBER,
    old_salary   NUMBER,
    new_salary   NUMBER,
    updated_by   VARCHAR2(30),
    update_time  DATE
);

CREATE OR REPLACE TRIGGER audit_salary_update
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    INSERT INTO emp_audit(emp_id, old_salary, new_salary,
                          updated_by, update_time)
    VALUES (
        :OLD.employee_id,
        :OLD.salary,
        :NEW.salary
        SYS_CONTEXT('USERENV', 'SESSION_USER'),
        SYSDATE
    );
END;
/
```

Demonstration

```
UPDATE employees
SET salary = salary + 2000
WHERE employee_id = 101;

SELECT * FROM emp_audit;
-- Shows old & new salary + username + timestamp
```

4 DDL Trigger Examples

4.1 Logging Schema Changes

```
CREATE TABLE ddl_log (
    username      VARCHAR2(30),
    operation     VARCHAR2(30),
    obj_type      VARCHAR2(30),
    obj_name      VARCHAR2(30),
    event_date    DATE
);

CREATE OR REPLACE TRIGGER log_schema_changes
AFTER DDL ON SCHEMA
BEGIN
    INSERT INTO ddl_log(username, operation, obj_type,
                        obj_name, event_date)
    VALUES (
        SYS_CONTEXT('USERENV','SESSION_USER'),
        ORA_SYSEVENT,
        ORA_DICT_OBJ_TYPE,
        ORA_DICT_OBJ_NAME,
        SYSDATE
    );
END;
/
```

Demonstration

```
CREATE TABLE demo_table(id NUMBER);
ALTER TABLE demo_table ADD name VARCHAR2(20);
DROP TABLE demo_table;

SELECT * FROM ddl_log;
-- Displays all CREATE/ALTER/DROP events
```

5 System Trigger Examples

5.1 Capture Login Information

```
CREATE TABLE user_login_log (
    username      VARCHAR2(30),
    login_time    DATE,
    ip_address    VARCHAR2(50)
```

```
);

CREATE OR REPLACE TRIGGER capture_user_login
AFTER LOGON ON DATABASE
BEGIN
    INSERT INTO user_login_log(username, login_time, ip_address)
    VALUES (
        SYS_CONTEXT('USERENV','SESSION_USER'),
        SYSDATE,
        SYS_CONTEXT('USERENV','IP_ADDRESS')
    );
END;
/
```

Demonstration

```
-- Simply log out and log in again.

SELECT * FROM user_login_log;
-- Shows username, login time, IP address
```

5.2 Email Notification Trigger for Privileged Users

```
CREATE OR REPLACE TRIGGER notify_dba_login
AFTER LOGON ON DATABASE
DECLARE
    v_user VARCHAR2(30);
BEGIN
    v_user := SYS_CONTEXT('USERENV', 'SESSION_USER');

    IF v_user IN ('SYS', 'SYSTEM') THEN
        UTL_MAIL.send(
            sender     => 'admin@database.com',
            recipients => 'dba_team@database.com',
            subject    => 'Privileged User Login Alert',
            message    => 'User ' || v_user ||
                          ' logged into the database.'
        );
    END IF;
END;
/
```

Demonstration

```
-- Login as SYS or SYSTEM
-- Email is sent automatically
```

5.3 Setting Time Zone Automatically

```
CREATE OR REPLACE TRIGGER set_user_timezone
AFTER LOGON ON DATABASE
BEGIN
    EXECUTE IMMEDIATE
        'ALTER SESSION SET TIME_ZONE = ''+05:00''';
END;
/
```

Demonstration

```
-- After login:
SELECT SESSIONTIMEZONE FROM dual;
-- Output: +05:00
```

6 Instead-of Trigger Examples (For Views)

6.1 Simple Insert Instead-of Trigger

```
CREATE VIEW emp_view AS
SELECT employee_id, first_name, salary
FROM employees;

CREATE OR REPLACE TRIGGER insert_emp_view
INSTEAD OF INSERT ON emp_view
FOR EACH ROW
BEGIN
    INSERT INTO employees(employee_id, first_name, salary)
    VALUES(:NEW.employee_id, :NEW.first_name, :NEW.salary);
END;
/
```

Demonstration

```
INSERT INTO emp_view VALUES (201, 'Hassan', 40000);

SELECT * FROM employees WHERE employee_id = 201;
-- Record successfully inserted through VIEW
```

7 More Instead-of Trigger Examples

7.1 1. Insert into a Multi-table View

```
CREATE VIEW student_course_view AS
SELECT s.student_id, s.name, c.course_name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;

CREATE OR REPLACE TRIGGER insert_student_course
INSTEAD OF INSERT ON student_course_view
FOR EACH ROW
BEGIN
    INSERT INTO students(student_id, name)
    VALUES(:NEW.student_id, :NEW.name);

    INSERT INTO enrollments(student_id, course_id)
    VALUES(:NEW.student_id,
           (SELECT course_id FROM courses
            WHERE course_name = :NEW.course_name));
END;
/
```

Demonstration

```
INSERT INTO student_course_view
VALUES (1, 'Ali', 'Database␣Systems');

SELECT * FROM students;
SELECT * FROM enrollments;
```

7.2 2. Updating a Complex Joined View

```
CREATE VIEW emp_dept_view AS
SELECT e.employee_id, e.first_name, d.department_name
FROM employees e
JOIN departments d
ON e.department_id = d.department_id;

CREATE OR REPLACE TRIGGER update_emp_dept
INSTEAD OF UPDATE ON emp_dept_view
FOR EACH ROW
BEGIN
    UPDATE employees
```

```
    SET first_name = :NEW.first_name
    WHERE employee_id = :OLD.employee_id;

    UPDATE departments
    SET department_name = :NEW.department_name
    WHERE department_name = :OLD.department_name;
END;
/
```

Demonstration

```
UPDATE emp_dept_view
SET first_name='Ahmad', department_name='Finance'
WHERE employee_id = 101;
```

7.3 3. DELETE Trigger on Joined View

```
CREATE VIEW order_details_view AS
SELECT o.order_id, o.order_date, c.customer_name
FROM orders o
JOIN customers c USING(customer_id);

CREATE OR REPLACE TRIGGER delete_order_details
INSTEAD OF DELETE ON order_details_view
FOR EACH ROW
BEGIN
    DELETE FROM orders WHERE order_id = :OLD.order_id;
    DELETE FROM customers
    WHERE customer_name = :OLD.customer_name;
END;
/
```

Demonstration

```
DELETE FROM order_details_view
WHERE order_id = 50;
```

7.4 4. FULL CRUD Instead-of Trigger

```
CREATE VIEW product_supplier_view AS
SELECT p.product_id, p.product_name,
       s.supplier_name, s.contact
FROM products p
```

```
JOIN suppliers s
ON p.supplier_id = s.supplier_id;

CREATE OR REPLACE TRIGGER product_supplier_crud
INSTEAD OF INSERT OR UPDATE OR DELETE
ON product_supplier_view
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO suppliers(supplier_name, contact)
        VALUES(:NEW.supplier_name, :NEW.contact);

        INSERT INTO products(product_id, product_name, supplier_id)
        VALUES(:NEW.product_id, :NEW.product_name,
               (SELECT supplier_id FROM suppliers
                WHERE supplier_name = :NEW.supplier_name));

    ELSIF UPDATING THEN
        UPDATE products
        SET product_name = :NEW.product_name
        WHERE product_id = :OLD.product_id;

        UPDATE suppliers
        SET supplier_name = :NEW.supplier_name,
            contact = :NEW.contact
        WHERE supplier_name = :OLD.supplier_name;

    ELSIF DELETING THEN
        DELETE FROM products
        WHERE product_id = :OLD.product_id;
    END IF;
END;
/
```

Demonstration

```
-- INSERT
INSERT INTO product_supplier_view
VALUES (10, 'Mouse', 'ABC Suppliers', '0300-1111111');

-- UPDATE
UPDATE product_supplier_view
SET product_name='Wireless Mouse'
WHERE product_id=10;

-- DELETE
DELETE FROM product_supplier_view
```

```
WHERE product_id=10;
```

7.5 5. Instead-of Trigger for Aggregated View

```
CREATE VIEW dept_salary_view AS
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;

CREATE OR REPLACE TRIGGER insert_dept_salary
INSTEAD OF INSERT ON dept_salary_view
FOR EACH ROW
BEGIN
    INSERT INTO employees(department_id, salary)
    VALUES(:NEW.department_id, :NEW.total_salary);
END;
/
```

Demonstration

```
INSERT INTO dept_salary_view
VALUES (50, 90000);

SELECT * FROM employees WHERE department_id=50;
```

8 Database Startup and Shutdown Triggers

```
CREATE TABLE system_events (
    event_type   VARCHAR2(30),
    event_time   DATE
);

CREATE OR REPLACE TRIGGER log_startup
AFTER STARTUP ON DATABASE
BEGIN
    INSERT INTO system_events VALUES ('STARTUP', SYSDATE);
END;
/
```

```
CREATE OR REPLACE TRIGGER log_shutdown
BEFORE SHUTDOWN ON DATABASE
BEGIN
    INSERT INTO system_events VALUES ('SHUTDOWN', SYSDATE);
```

```
END;
/
```

Demonstration

```
-- Shut down and restart database
SELECT * FROM system_events;
```

9 DML Trigger Tasks

9.1 1. Automatically Insert Bonus into employee_bonus Table

```
CREATE TABLE employee_bonus (
    emp_id         NUMBER,
    bonus_amount   NUMBER,
    bonus_date     DATE DEFAULT SYSDATE
);

CREATE OR REPLACE TRIGGER trg_add_bonus
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO employee_bonus(emp_id, bonus_amount)
    VALUES(:NEW.employee_id, :NEW.salary * 0.10);
END;
/
```

Demonstration:

```
INSERT INTO employees(employee_id, first_name, salary)
VALUES (101, 'Alice', 5000);

-- Automatically inserts 500 (10% of 5000) into employee_bonus table
SELECT * FROM employee_bonus;
```

9.2 2. Prevent Salary Update Above 10,000

```
CREATE OR REPLACE TRIGGER trg_salary_limit
BEFORE UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary > 10000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary cannot exceed
           10,000.');
```

```
    END IF;
END;
/
```

Demonstration:

```
UPDATE employees SET salary = 15000 WHERE employee_id = 101;
-- Raises error: Salary cannot exceed 10,000.
```

9.3 3. Log Deleted Records into Deleted_Employees_Log

```
CREATE TABLE Deleted_Employees_Log (
    emp_id      NUMBER,
    first_name  VARCHAR2(50),
    last_name   VARCHAR2(50),
    salary      NUMBER,
    deleted_at  DATE,
    deleted_by  VARCHAR2(50)
);

CREATE OR REPLACE TRIGGER trg_log_delete
BEFORE DELETE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO Deleted_Employees_Log
    VALUES(:OLD.employee_id, :OLD.first_name, :OLD.last_name,
           :OLD.salary, SYSDATE, USER);
END;
/
```

Demonstration:

```
DELETE FROM employees WHERE employee_id = 101;
-- Record is inserted into Deleted_Employees_Log automatically
SELECT * FROM Deleted_Employees_Log;
```

9.4 4. Log Old and New Salary on Update

```
CREATE TABLE Salary_Change_Log (
    emp_id      NUMBER,
    old_salary  NUMBER,
    new_salary  NUMBER,
    changed_on  DATE,
    changed_by  VARCHAR2(50)
);

CREATE OR REPLACE TRIGGER trg_salary_history
```

```
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    INSERT INTO Salary_Change_Log
    VALUES(:OLD.employee_id, :OLD.salary, :NEW.salary, SYSDATE, USER
       );
END;
/
```

Demonstration:

```
UPDATE employees SET salary = 6000 WHERE employee_id = 101;
-- Changes logged automatically
SELECT * FROM Salary_Change_Log;
```

10 DDL Trigger Tasks

10.1 1. Log Table Creation Events

```
CREATE TABLE Audit_Log (
    object_name   VARCHAR2(100),
    event_type    VARCHAR2(50),
    created_by    VARCHAR2(50),
    event_time    DATE
);

CREATE OR REPLACE TRIGGER trg_log_table_creation
AFTER CREATE ON SCHEMA
BEGIN
    IF ORA_DICT_OBJ_TYPE = 'TABLE' THEN
        INSERT INTO Audit_Log
        VALUES (ORA_DICT_OBJ_NAME, 'TABLE␣CREATED', USER, SYSDATE);
    END IF;
END;
/
```

10.2 2. Prevent ALTER on Employees After Hours

```
CREATE OR REPLACE TRIGGER trg_prevent_alter
BEFORE ALTER ON employees
BEGIN
    IF TO_CHAR(SYSDATE, 'HH24') NOT BETWEEN '08' AND '18' THEN
        RAISE_APPLICATION_ERROR(-20002,
            'ALTER␣is␣not␣allowed␣after␣business␣hours␣(8␣AM␣   ␣6␣
               PM).');
```

```
    END IF;
END;
/
```

10.3 3. Log DROP Operations

```
CREATE TABLE Drop_Log (
    object_name  VARCHAR2(100),
    dropped_by   VARCHAR2(50),
    dropped_at   DATE
);

CREATE OR REPLACE TRIGGER trg_log_drop
AFTER DROP ON DATABASE
BEGIN
    INSERT INTO Drop_Log
    VALUES(ORA_DICT_OBJ_NAME, USER, SYSDATE);
END;
/
```

10.4 4. Prevent Dropping Audit_Log Table

```
CREATE OR REPLACE TRIGGER trg_protect_auditlog
BEFORE DROP ON DATABASE
BEGIN
    IF ORA_DICT_OBJ_NAME = 'AUDIT_LOG' THEN
        RAISE_APPLICATION_ERROR(-20003, 'Audit_Log table cannot be
           dropped.');
    END IF;
END;
/
```

11 System / Database Triggers

11.1 1. Log Database Startup

```
CREATE TABLE System_Logs (
    event_type   VARCHAR2(40),
    event_time   DATE
);

CREATE OR REPLACE TRIGGER trg_db_startup
AFTER STARTUP ON DATABASE
```

```
BEGIN
    INSERT INTO System_Logs VALUES ('DATABASE STARTED', SYSDATE);
END;
/
```

11.2 2. Track Failed Login Attempts

```
CREATE TABLE Failed_Logins (
    username    VARCHAR2(40),
    event_time  DATE,
    os_user     VARCHAR2(40)
);

CREATE OR REPLACE TRIGGER trg_failed_login
AFTER SERVERERROR ON DATABASE
BEGIN
    IF IS_SERVERERROR(1017) THEN
        INSERT INTO Failed_Logins
        VALUES (USER, SYSDATE, SYS_CONTEXT('USERENV','OS_USER'));
    END IF;
END;
/
```

11.3 3. Log Successful Logout + Session Duration

```
CREATE TABLE User_Activity_Log (
    username         VARCHAR2(40),
    login_time       DATE,
    logout_time      DATE,
    session_duration NUMBER
);

CREATE OR REPLACE TRIGGER trg_logout_activity
BEFORE LOGOFF ON DATABASE
BEGIN
    INSERT INTO User_Activity_Log
    VALUES (
        USER,
        SYSDATE - (SYS_CONTEXT('USERENV','SESSION_LENGTH')/24/60),
        SYSDATE,
        SYS_CONTEXT('USERENV','SESSION_LENGTH')
    );
END;
/
```

12 Instead-of Triggers

12.1 1. Insert Into Employees + Departments via View

```
CREATE OR REPLACE VIEW emp_dept_view AS
SELECT e.employee_id, e.first_name, d.department_id, d.
   department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id;

CREATE OR REPLACE TRIGGER trg_io_insert_emp_dept
INSTEAD OF INSERT ON emp_dept_view
FOR EACH ROW
BEGIN
    INSERT INTO departments (department_id, department_name)
    VALUES (:NEW.department_id, :NEW.department_name);

    INSERT INTO employees (employee_id, first_name, department_id)
    VALUES (:NEW.employee_id, :NEW.first_name, :NEW.department_id);
END;
/
```

12.2 2. Prevent Salary Reduction ¿ 20% via View

```
CREATE OR REPLACE VIEW emp_salary_view AS
SELECT employee_id, salary
FROM employees;

CREATE OR REPLACE TRIGGER trg_io_salary_update
INSTEAD OF UPDATE OF salary ON emp_salary_view
FOR EACH ROW
BEGIN
    IF :NEW.salary < (:OLD.salary * 0.80) THEN
        RAISE_APPLICATION_ERROR(-20004,
            'Salary cannot be reduced by more than 20%.');
    ELSE
        UPDATE employees
        SET salary = :NEW.salary
        WHERE employee_id = :OLD.employee_id;
    END IF;
END;
/
```

13 Conclusion

This lab demonstrated the implementation of various PL/SQL triggers including DML, DDL, system-level, auditing, login monitoring, and multiple advanced instead-of triggers. Each trigger was tested using demonstration SQL statements.